

A PROJECT REPORT ON

Automatic Summarization of Legal Documents

SUBMITTED TO THE SAVITRIBAI PHULE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE

OF

BACHELOR OF ENGINEERING (Computer Engineering)

SUBMITTED BY

Aayush Goyal
Arnav Firke
Bhairu Gavhane
Shrinidhi Kulkarni

Exam No: B400050311
Exam No: B400050388
Exam No: B400050397
Exam No: B400050463



DEPARTMENT OF COMPUTER ENGINEERING
Pune Institute of Computer Technology
Dhankawadi, Pune - 411043

SAVITRIBAI PHULE PUNE UNIVERSITY
2024-2025



CERTIFICATE

This is to certify that the project report entitled

Automatic Summarization of Legal Documents

Submitted by

Aayush Goyal

Exam No: B400050311

Arnav Firke

Exam No: B400050388

Bhairu Gavhane

Exam No: B400050397

Shrinidhi Kulkarni

Exam No: B400050463

is a bonafide student of this institute and the work has been carried out by him/her under the supervision of **Prof. P. P. Joshi** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** (Computer Engineering).

Prof. P. P. Joshi
Internal Guide
Dept. of Computer Engg.

Dr. Geetanjali V. Kale
Head
Dept. of Computer Engg.

Dr. S. T. Gandhe
Principal
Pune Institute of Computer Technology

Place:Pune

Date:

ACKNOWLEDGEMENT

*It gives us great pleasure in presenting the project report on ‘**Automatic Summarization of Legal Documents**’.*

*We would like to take this opportunity to thank **Prof. P. P. Joshi** for giving us all the help and guidance we needed. We are truly grateful to her for her kind support. Her valuable suggestions were very helpful.*

*We are also grateful to **Dr. Geetanjali V. Kale**, Head of Computer Engineering Department, PICT for her indispensable support, suggestions.*

Aayush Goyal
Arnav Firke
Bhairu Gavhane
Shrinidhi Kulkarni
(B.E. Computer Engg.)

ABSTRACT

This study presents an innovative approach to summarizing Indian judicial case documents by fine-tuning a BART model with Low-Rank Adaptation (LoRA). The complexity and length of Indian legal texts pose significant challenges for summarization, necessitating tailored solutions. We introduce a chunking mechanism to effectively process lengthy documents within model constraints, ensuring contextual integrity. Our method leverages standard training objectives, optimized through a structured hyperparameter search, to balance semantic accuracy and summary coherence. Evaluated on a preprocessed dataset of Indian judicial cases, the approach demonstrates enhanced summary quality and relevance. This research advances the automation of legal text summarization, offering a scalable solution to improve efficiency in legal research and case analysis within the Indian legal system.

Contents

1	Introduction	1
1.1	Overview	2
1.2	Motivation	2
1.3	Problem Definition and Objectives	2
1.3.1	Project Definition	2
1.3.2	Project Objectives	3
1.4	Project Scope & Limitation	3
1.5	Methodologies of Problem solving	4
2	Literature Survey	5
2.1	Automatic Text Summarization Techniques	6
2.2	Legal Document Summarization Challenges	6
2.3	Existing Systems and Models	7
2.4	Key Research Studies	7
2.5	Gaps in Current Research	7
3	Software Requirements Specification	9
3.1	Assumptions and Dependencies	10
3.2	Functional Requirements	10
3.3	External Interface Requirements	11
3.3.1	User Interfaces	11
3.3.2	Hardware Interfaces	11
3.3.3	Software Interfaces	11
3.3.4	Communication Interfaces	11
3.4	Nonfunctional Requirements	11
3.4.1	Performance Requirements	11
3.4.2	Safety Requirements	12
3.4.3	Security Requirements	12
3.5	System Requirements	12
3.5.1	Database Requirements	12
3.5.2	Software Requirements (Platform Choice)	12

3.5.3	Hardware Requirements	13
3.6	Analysis Models: SDLC Model to be Applied	13
3.6.1	Justification for Choosing the Iterative Waterfall Model . . .	13
3.6.2	Phases of the SDLC Model in the Project	13
4	System Design	15
4.1	System Architecture	16
4.1.1	Input Layer	16
4.1.2	BART Encoder-Decoder Architecture	17
4.1.3	LoRA Fine-Tuning	17
4.1.4	Summarization Training	17
4.1.5	Output Layer	17
4.1.6	User Interface (UI)	18
4.1.7	Storage and Retrieval	18
4.2	Mathematical Model:	18
4.2.1	Model Architecture	18
4.2.2	Training Objective	19
4.2.3	Inference	19
4.2.4	Relevance to Legal Text Summarization	19
4.3	Data Flow Diagrams	20
4.4	Entity Relationship Diagram	21
4.5	UML Diagrams	22
4.6	Sequence Diagram	24
5	Project Implementation	25
5.1	Overview of Project Modules	26
5.2	Tools and Technologies Used	26
5.3	Algorithm Details	27
5.3.1	Text Chunking and Preprocessing	27
5.3.2	Dataset Construction	27
5.3.3	Model Initialization with LoRA	28
5.3.4	Training Procedure	28
5.3.5	Evaluation Metrics	29
5.3.6	Hyperparameter Tuning	29
6	Project Plan	30
6.1	Project Estimates	31
6.1.1	Reconciled Estimates	31
6.2	Risk Management	32
6.2.1	Risk Identification	32
6.2.2	Risk Analysis	32

6.2.3	Risk Mitigation, Monitoring, and Management	32
6.3	Project Schedule	33
6.3.1	Project Task Set	33
6.3.2	Timeline Chart	33
6.4	Team Organization	33
6.4.1	Team Structure	33
6.4.2	Management Reporting and Communication	34
7	Software Testing	35
7.1	Type of Testing	36
7.2	Test Cases & Test Results	36
8	Results	37
8.1	Outcomes	38
8.2	Screen Shots	38
9	Conclusions	40
9.1	Conclusion	41
9.2	Future Work	41
9.3	Applications	41
10	Appendix	43
10.1	Appendix A	44
10.1.1	Problem Statement Overview	44
10.1.2	Feasibility via Satisfiability	44
10.1.3	Complexity Classification	45
10.1.4	Modern Algebraic Insight	46
10.1.5	Practical Implication	46
10.2	Appendix B	47
10.2.1	Survey Paper	47
10.2.2	Implementation Paper	47
10.3	Appendix C	48
11	References	51

List of Figures

3.1	Functional Requirements	10
4.1	BART Architecture Diagram	16
4.2	Data Flow Diagram	20
4.3	Activity Diagram	21
4.4	Activity Diagram	22
4.5	UseCase Diagram	22
4.6	Class Diagram	23
4.7	Sequence Diagram	24
8.1	Screenshot of the final results	38
8.2	Screenshot of the final results	39
8.3	Screenshot of the final results	39

CHAPTER 1

INTRODUCTION

1.1 Overview

This project focuses on the development of an AI model for the automatic summarization of legal documents using Natural Language Processing (NLP) and machine learning techniques. Legal texts are often lengthy, complex, and filled with domain-specific jargon, making manual summarization inefficient and error-prone. The proposed system aims to address this challenge by generating accurate and concise summaries that preserve the core information from various types of legal documents, such as contracts, agreements, and judicial opinions. The end goal is to assist legal professionals, businesses, and general users in understanding legal content more efficiently and effectively.

1.2 Motivation

Legal documents such as contracts, agreements, case law, and court rulings are often lengthy and complex, making it challenging for legal professionals and clients to review them quickly. With the growing volume of legal data, there's a pressing need for tools that can automatically summarize such documents without missing crucial information. Summarization helps professionals reduce time spent manually reading through documents, enabling them to focus on higher-level tasks. Legal document summarization can be useful in various sectors, including law firms, corporate legal departments, and public institutions. Moreover, the rise of AI and Natural Language Processing (NLP) has enabled the development of models that can analyze and extract essential information from legal texts. This motivates the use of NLP techniques in addressing the complexities of legal documentation, such as intricate language, structure, and terminology.

1.3 Problem Definition and Objectives

1.3.1 Project Definition

Legal documents are typically lengthy, highly structured, and often include complex sentence constructions and domain-specific terminology. These characteristics make manual summarization both time-consuming and prone to human error. Furthermore, understanding and interpreting such documents can be challenging for individuals without legal expertise.

This project addresses the challenge of automatically summarizing legal documents using advanced Natural Language Processing (NLP) and machine learning techniques. The primary focus is to develop a system that can efficiently process

Indian judicial legal documents and generate concise, accurate summaries that retain essential information.

1.3.2 Project Objectives

- Develop a robust and scalable summarization system tailored for legal documents.
- Utilize state-of-the-art NLP models, specifically fine-tuned for legal language processing.
- Ensure the output summaries are accurate, cohesive, and understandable to both legal professionals and laypersons.
- Design a user-friendly interface for uploading documents and viewing summaries in real-time.
- Support multiple document formats and maintain flexibility to expand to multilingual capabilities in the future.

1.4 Project Scope & Limitation

The scope of this project is to develop an automated legal document summarization tool that can process large volumes of legal texts and generate concise, accurate summaries. The system will cater to the following user groups:

- **Legal Professionals:** Lawyers and legal researchers can use this tool to summarize long contracts, case laws, and statutes, thereby saving time and improving efficiency.
- **Businesses:** Companies involved in contract negotiations can quickly understand the key points of legal documents without requiring extensive legal expertise.
- **General Users:** Individuals seeking to comprehend complex legal terms or agreements can benefit from simplified summaries generated by the tool.

The system will employ natural language processing (NLP) models that are specifically trained on legal texts to ensure domain-specific accuracy and relevance. The final output will be a user-friendly application that allows users to upload legal documents and receive automated summaries in real time.

1.5 Methodologies of Problem solving

- **Problem Identification:** Clearly defining the issue is critical. Analyze symptoms, context, and stakeholders to pinpoint the root cause. Use tools like problem statements or fishbone diagrams to structure the process, ensuring a precise understanding of the challenge before proceeding to solution development.
- **Data Collection:** Gather comprehensive data to inform the problem's scope. Collect quantitative metrics, qualitative feedback, and historical records. Engage stakeholders for insights and verify data reliability to ensure a robust foundation for analysis, enabling informed decision-making in subsequent problem-solving stages.
- **Solution Generation:** Brainstorm diverse solutions to address the problem. Encourage creative thinking, combining innovative and practical ideas. Use techniques like mind mapping or SCAMPER to explore options. Ensure solutions align with objectives, considering feasibility, resources, and constraints for optimal outcomes.
- **Evaluation and Selection:** Assess solutions against criteria such as impact, cost, and scalability. Use decision matrices or cost-benefit analysis to compare options. Prioritize solutions balancing short-term gains and long-term sustainability, selecting the most effective approach to address the problem comprehensively and efficiently.
- **Implementation:** Execute the chosen solution with a detailed plan. Define timelines, assign responsibilities, and allocate resources. Monitor progress through milestones, addressing challenges promptly. Communicate with stakeholders to ensure alignment, adapting the plan as needed to achieve successful deployment and desired outcomes.
- **Review and Optimization:** Evaluate the solution's effectiveness post-implementation. Collect feedback, measure outcomes against goals, and identify gaps. Refine the approach through iterative improvements, incorporating lessons learned. Optimize processes to enhance future problem-solving, ensuring sustained success and adaptability to evolving challenges.

CHAPTER 2

LITERATURE SURVEY

2.1 Automatic Text Summarization Techniques

There are two major approaches to text summarization: extractive summarization and abstractive summarization.

- **Extractive Summarization:** This technique selects sentences or phrases directly from the original text to form a summary. The selection is based on factors like sentence relevance, word frequency, and position in the document. While early summarization models favored this approach for its simplicity, the downside is a lack of cohesion due to unaltered sentence structure.
 - **LexRank:** A graph-based algorithm that identifies important sentences using PageRank-like techniques.
 - **TextRank:** A similar graph-based model focusing on keyword extraction for summary generation.
- **Abstractive Summarization:** This method generates new sentences that may not be present in the original document. It paraphrases the content for a more fluent and cohesive summary. This approach requires deeper semantic understanding and has recently been enhanced by Transformer-based models.
 - **Seq2Seq Models:** Based on Recurrent Neural Networks (RNNs), these generate summaries by learning sentence structure.
 - **Transformer Models** (e.g., BERT, GPT): Pre-trained language models that use attention mechanisms for generating more contextually accurate summaries.

2.2 Legal Document Summarization Challenges

Legal documents pose unique challenges compared to general text:

- **Legal Jargon:** Abundant use of domain-specific terminology that requires precise interpretation.
- **Document Structure:** Typically follows a rigid structure with sections like preamble, definitions, clauses, and annexes.
- **Ambiguity:** Slight variations in wording can drastically change legal interpretations.
- **Data Scarcity:** There is a lack of large, annotated legal datasets for training summarization models.

2.3 Existing Systems and Models

Several tools and research efforts have been developed in this area:

- **CaseSummarizer:** Uses extractive summarization with sentence scoring algorithms to summarize judicial decisions.
- **SUMMA:** A multilingual summarization system that handles legal and news content using both extractive and abstractive techniques.
- **Contract Simplification Tools:** Tools such as Litera and Kira focus on clause extraction and summarization for contract analysis.

2.4 Key Research Studies

- *A Neural Network-Based Approach for Summarizing Legal Texts:* Uses an encoder-decoder architecture with attention mechanisms for abstractive summarization.
- *Summarizing Contracts Using Deep Learning:* Explores the use of large legal datasets and fine-tuned BART models.
- *Plain English Summarization of Contracts:* Focuses on simplifying unilateral contracts using paired datasets obtained from TOS-DR and TLDRLegal.
- *Summarization of Commercial Contracts:* Employs Latent Semantic Analysis (LSA) and TextRank, with options for topic-specific summarization. LSA showed the best F-measure scores among evaluated methods.

2.5 Gaps in Current Research

- Limited support for multilingual legal document summarization.
- Lack of context-aware models capable of understanding legal nuances.
- Few solutions provide real-time summarization, which is essential for courts and legal firms.
- Limited exploration of abstractive methods due to dataset constraints.
- A need exists for larger corpora with human-written summaries to improve supervised methods.

This literature review forms the foundation for the model selection and development approach in this project. Our work aims to build on these insights and address the existing gaps.

CHAPTER 3

**SOFTWARE REQUIREMENTS
SPECIFICATION**

3.1 Assumptions and Dependencies

Assumptions:

- The system assumes that the uploaded documents are in a readable format (e.g., PDF, Word).
- It assumes that the legal texts provided are in English.
- Users will require basic internet access to upload documents and view summaries.

Dependencies:

- The system depends on external libraries (e.g., NLP toolkits like Hugging Face, Spacy) to process and summarize legal documents.
- It also depends on cloud computing resources for model training and document processing.

3.2 Functional Requirements

System Feature	Description	Input	Output	Criticality
Upload and Document Processing	The system should allow users to upload documents in different formats (PDF, DOCX, TXT).	The user provides a legal document.	The system converts the document into a readable format, processes it using NLP techniques, and displays a summarized version.	High
Legal Text Summarization	The system summarizes legal documents while retaining key points like terms, parties involved, and obligations.	The text of the legal document.	A shortened summary of the document that captures the key legal information.	High
Summary Customization	Users should be able to select the level of summarization required, such as brief, medium, or detailed.	User's preference for summary length.	A summary matching the selected level of detail.	Medium
Legal Jargon Explanation	The system should provide explanations for legal jargon or complex terms found in the documents.	The legal document text.	Simplified explanations of key legal terms.	Medium

Figure 3.1: Functional Requirements

3.3 External Interface Requirements

3.3.1 User Interfaces

3.3.1.1 Interface Design

A clean and intuitive user interface (UI) that allows users to upload documents, choose summarization preferences, and view the summary.

Accessibility: The interface should be accessible to non-technical users and offer clear instructions.

3.3.2 Hardware Interfaces

Input Devices: Standard input devices like keyboard, mouse, or touch-enabled devices for document upload.

Output Devices: The summarized text will be presented on standard displays (laptops, mobile devices, desktops).

3.3.3 Software Interfaces

The system will integrate with:

- NLP libraries (e.g., TensorFlow, PyTorch, Hugging Face)
- Cloud platforms for scalable processing (e.g., AWS, Google Cloud)
- Document readers to extract text from uploaded PDFs or Word files

3.3.4 Communication Interfaces

The system should support internet-based communication protocols like HTTP/HTTPS for transferring data between the client and server.

3.4 Nonfunctional Requirements

3.4.1 Performance Requirements

- The system should process and summarize documents within a reasonable time frame, ideally under 10-15 seconds for medium-length documents.
- It should handle multiple concurrent users efficiently, with minimal delays in processing.

3.4.2 Safety Requirements

- The system should ensure that users' legal documents are processed securely and not shared with third parties.
- Legal documents must be deleted from the system after summarization to maintain confidentiality.

3.4.3 Security Requirements

- The system should use encryption (SSL/TLS) for data transmission to protect sensitive legal documents.
- User authentication (username and password) should be in place to prevent unauthorized access.
- Role-based access control (RBAC) should be implemented to limit access to specific system features.

3.5 System Requirements

3.5.1 Database Requirements

- A database system (e.g., MongoDB, PostgreSQL) will be used to store user information, legal documents (temporarily), and summaries.
- The database should support fast read/write operations and be scalable to handle large document volumes.

3.5.2 Software Requirements (Platform Choice)

- **Backend:** Node.js, Express.js
- **Frontend:** React.js
- **NLP Processing:** Python with libraries like Hugging Face, TensorFlow, or PyTorch
- **Database:** MongoDB for document storage

3.5.3 Hardware Requirements

- **Development Environment:** A powerful workstation with at least 16GB RAM and a multi-core processor
- **Deployment:** Cloud servers (AWS or GCP) to support large-scale document processing

3.6 Analysis Models: SDLC Model to be Applied

For the development of the automatic summarization system for legal text, the **Iterative Waterfall Model** from the Software Development Life Cycle (SDLC) framework is chosen. This model is suitable for projects that benefit from structured phases while allowing for refinement through iteration. Given the complexity and domain-specific nature of legal text, an iterative approach helps ensure accuracy, compliance, and user satisfaction.

3.6.1 Justification for Choosing the Iterative Waterfall Model

- **Structured Phase Division:** The project involves clear phases such as requirements gathering, data preprocessing, model development, UI design, and testing. The waterfall model supports such segmentation.
- **Iteration for Improvements:** Unlike the rigid traditional waterfall model, the iterative version allows feedback from each phase to refine previous phases, which is critical for tuning NLP models and handling diverse legal document types.
- **Early Risk Detection:** This model supports identifying risks (e.g., data availability, model performance) early in the development cycle, enabling better mitigation strategies.
- **Stakeholder Validation:** Regular reviews after each iteration allow legal domain experts to validate intermediate summaries, ensuring accuracy and legal compliance.

3.6.2 Phases of the SDLC Model in the Project

1. **Requirement Analysis:** Define system objectives, user needs, legal compliance constraints, and scope. Involves collaboration with legal professionals to identify key document types and summary expectations.

2. **System Design:** Architect the system including data flow, selection of pre-trained language models, model architecture, and user interface layout.
3. **Implementation:** Fine-tune transformer models such as BART or GPT on legal datasets. Build the backend pipeline for data input, model inference, and summary generation.
4. **Testing:** Perform functional, accuracy, and compliance testing. Include unit tests for model components and system integration tests.
5. **Deployment:** Launch the system in a cloud environment with user access via a web interface. Set up monitoring tools for performance and usage tracking.
6. **Feedback and Iteration:** Based on user feedback and model evaluation, iterate over the design and implementation to improve summarization quality and user experience.

This model balances predictability with adaptability, making it ideal for complex NLP projects like legal text summarization where precision and compliance are non-negotiable.

CHAPTER 4

SYSTEM DESIGN

System design refers to the detailed breakdown of how the components of the project work together to achieve the functionality laid out in the SRS. This includes the architectural layout, flow of data, entity relationships, and various design models used during the development process.

4.1 System Architecture

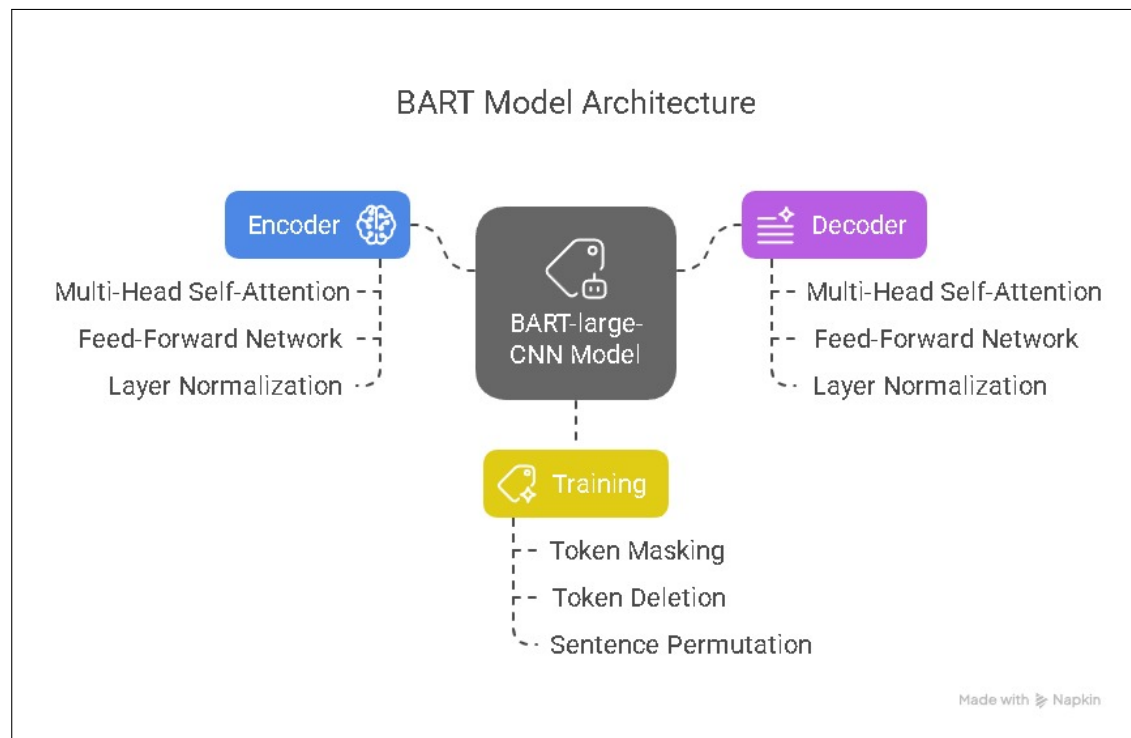


Figure 4.1: BART Architecture Diagram

4.1.1 Input Layer

The system accepts Indian judicial case documents, such as court rulings and legal judgments, as input. These texts undergo preprocessing via tokenization, where documents are split into sentences using NLTK’s tokenizer and further tokenized into words with BART’s tokenizer from Hugging Face. To manage lengthy documents, a chunking mechanism divides texts into segments of up to 1024 tokens for documents and 512 for summaries, preserving contextual integrity. Each chunk

is converted into token embeddings, capturing semantic meaning, with attention masks to handle padding and ensure the model focuses on relevant tokens.

4.1.2 BART Encoder-Decoder Architecture

The tokenized chunks are processed through BART-large-CNN’s sequence-to-sequence architecture, comprising an encoder and decoder. The encoder uses multiple transformer layers with multi-head self-attention to capture relationships across legal text, such as connections between clauses. The encoded representations feed into the decoder, which generates summary tokens autoregressively, attending to both the input and previously generated tokens. Feed-forward neural networks in each layer enhance contextual processing, with residual connections and normalization ensuring stable training.

4.1.3 LoRA Fine-Tuning

Low-Rank Adaptation (LoRA) is applied to fine-tune BART efficiently for legal summarization. LoRA introduces low-rank matrices to the query and value projections of BART’s attention layers, adapting the model to Indian judicial texts with minimal parameter updates. This reduces memory and computational demands, enabling effective fine-tuning on domain-specific data while retaining BART’s pre-trained language capabilities. The approach optimizes performance in resource-limited settings, making it ideal for legal applications.

4.1.4 Summarization Training

The model is fine-tuned for abstractive summarization using a standard cross-entropy loss, guiding the decoder to generate concise summaries capturing key legal details, such as case outcomes and critical arguments. The training leverages paired document-summary chunks, with a prefix “summarize:” to signal the task. A linear scheduler with warmup and gradient accumulation ensures stable convergence, optimizing the model to produce coherent and informative summaries.

4.1.5 Output Layer

The decoder outputs a sequence of tokens forming the summary, generated via beam search with four beams to select high-probability sequences. The output tokens are decoded into natural language, producing a readable summary that condenses the judicial document’s essential information. The final summary is structured for clarity, aiding legal professionals in case analysis.

4.1.6 User Interface (UI)

The system features a user-friendly web interface where users can upload their legal documents. The summarized text is displayed within the interface, and users have the option to download or copy the summary. The UI may also offer features such as document preview and input area, a download button for summarized legal texts, and options to handle multiple file formats like PDF or DOCX.

4.1.7 Storage and Retrieval

The system includes a storage component that allows users to save summarized legal documents. These documents are stored in a secure database and can be retrieved later for further review. The storage layer ensures quick access and retrieval of previous summaries, aiding in legal research and compliance processes.

4.2 Mathematical Model:

BART (Bidirectional and Auto-Regressive Transformers) is a denoising autoencoder-based sequence-to-sequence model introduced by Facebook AI. It combines the strengths of BERT (which is bidirectional and trained using a masked language modeling objective) and GPT (which is autoregressive and generates text sequentially).

4.2.1 Model Architecture

BART is built using a Transformer-based encoder-decoder architecture, where:

- The **encoder** is similar to BERT and learns a bidirectional representation of the input.
- The **decoder** is similar to GPT and generates the output autoregressively.

Formally, for an input sequence $x = (x_1, x_2, \dots, x_n)$, the BART model estimates the conditional probability of the output sequence $y = (y_1, y_2, \dots, y_m)$ as:

$$P(y \mid x) = \prod_{t=1}^m P(y_t \mid y_{<t}, x; \theta) \quad (4.1)$$

where θ denotes the model parameters and $y_{<t}$ represents all previously generated tokens up to time step t .

4.2.2 Training Objective

BART is trained using a denoising objective, where a corrupted version of the input $\tilde{x}$ is passed into the encoder, and the decoder learns to reconstruct the original sequence x . The training loss is the negative log-likelihood of the original sequence:

$$\mathcal{L}(\theta) = - \sum_{t=1}^n \log P(x_t \mid x_{<t}, \tilde{x}; \theta) \quad (4.2)$$

BART applies various noise functions to corrupt the input x , such as:

- Token masking
- Token deletion
- Sentence permutation
- Text infilling (span-based masking)

These corruption techniques help the model generalize better by learning to reconstruct a wide range of corrupted inputs.

4.2.3 Inference

During inference, the encoder processes the full input sequence and generates contextual embeddings, which are then fed into the decoder. The decoder generates the target sequence one token at a time using the previously generated tokens as context:

$$\hat{y}_t = \arg \max_{v \in V} P(y_t = v \mid \hat{y}_{<t}, x) \quad (4.3)$$

where V is the vocabulary of the model.

4.2.4 Relevance to Legal Text Summarization

BART is well-suited for abstractive summarization tasks such as legal document summarization because:

- It captures bidirectional context in the input, which is crucial for understanding complex legal language.
- Its autoregressive decoder generates fluent, coherent summaries.
- It handles long-range dependencies effectively, often present in legal texts.

4.3 Data Flow Diagrams

A description of each major software function, along with data flow (structured analysis) or class hierarchy (Analysis Class diagram with class description for object oriented system) is presented.

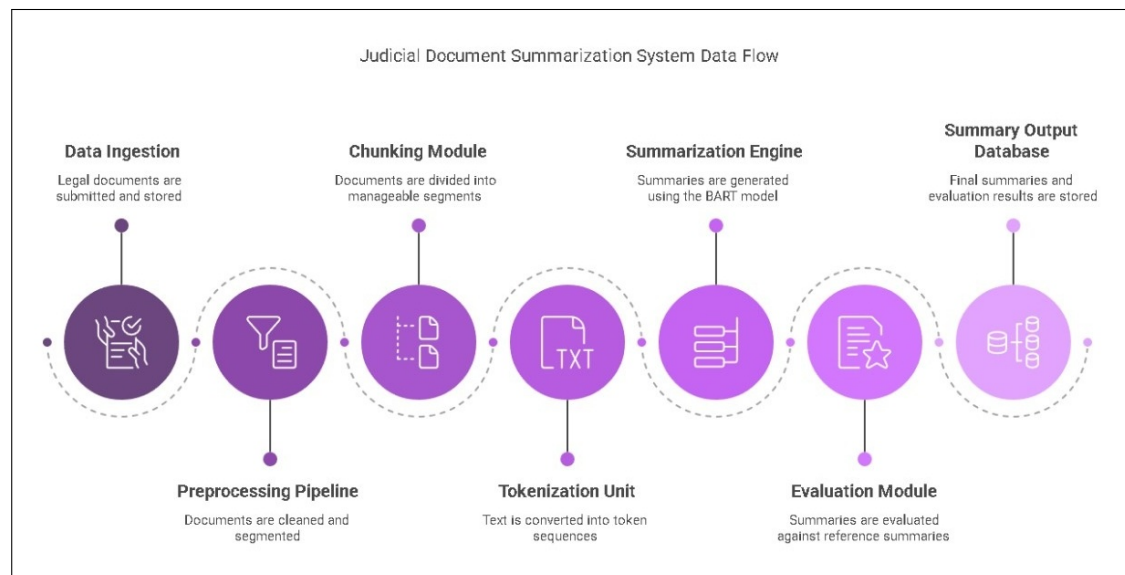


Figure 4.2: Data Flow Diagram

4.4 Entity Relationship Diagram

Here, we have shown how two different entities will interact using the system architecture.

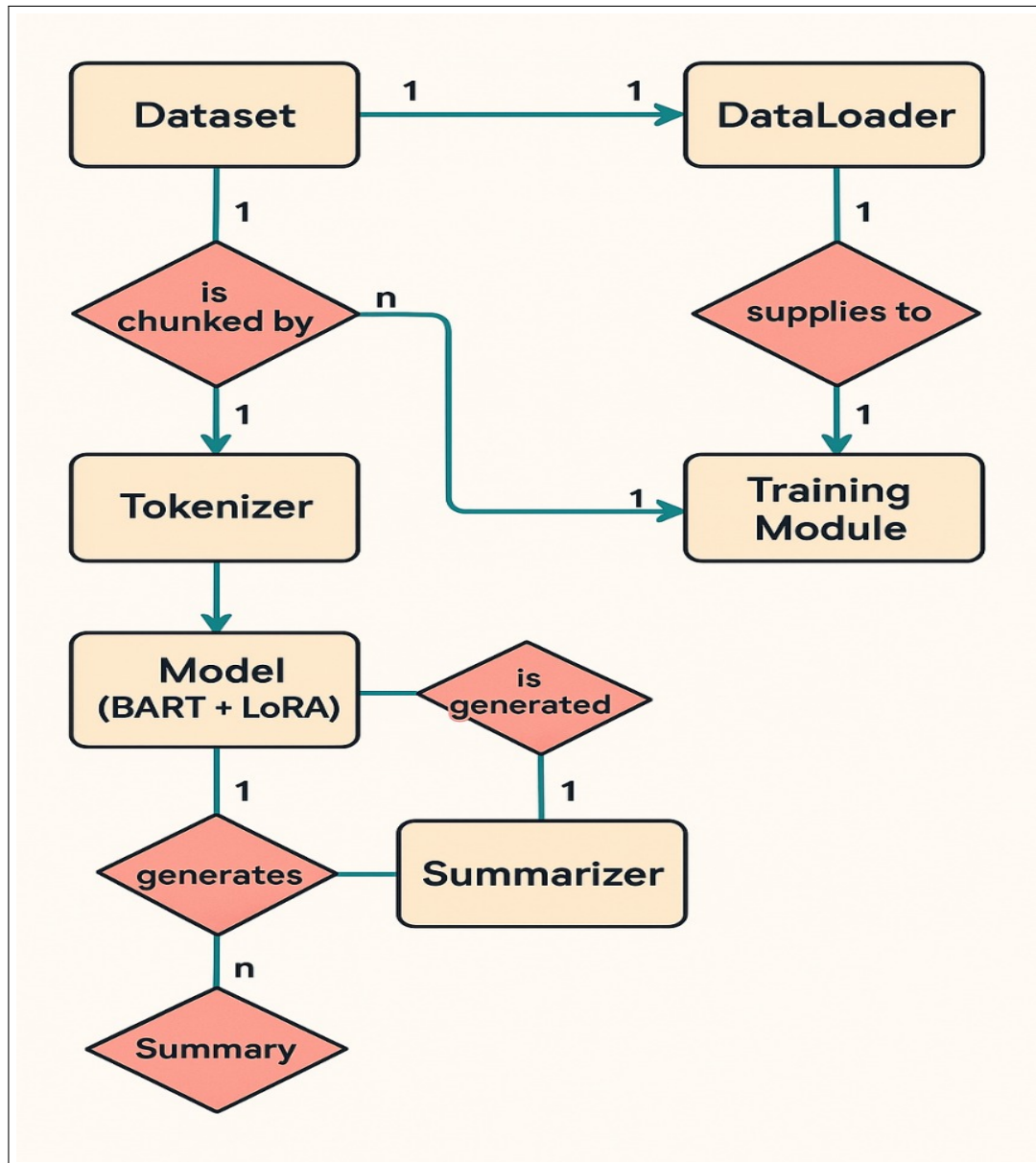


Figure 4.3: Activity Diagram

4.5 UML Diagrams

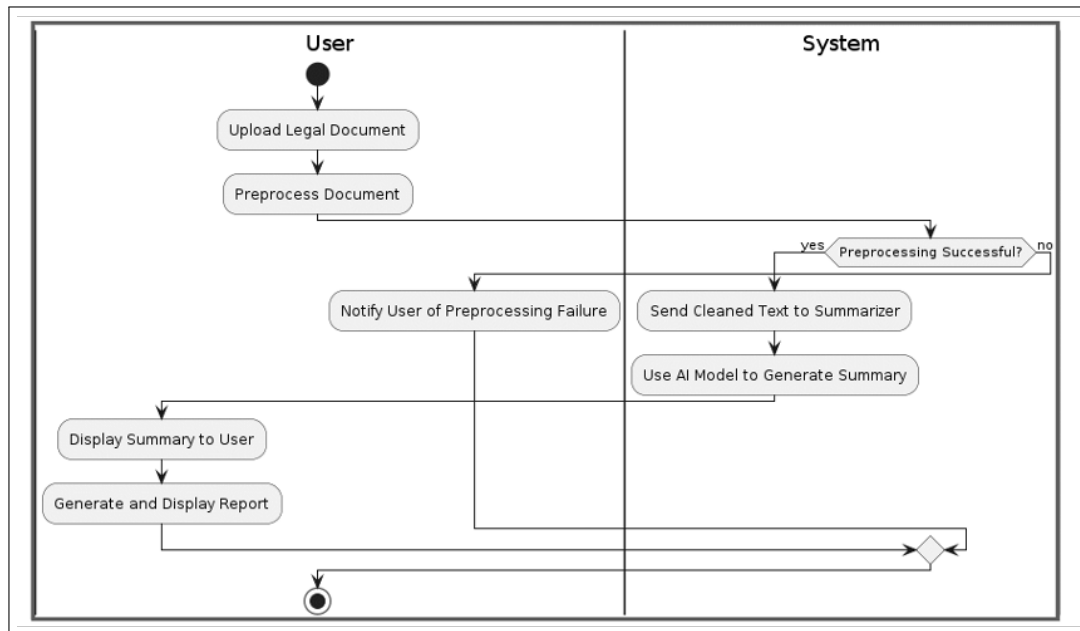


Figure 4.4: Activity Diagram

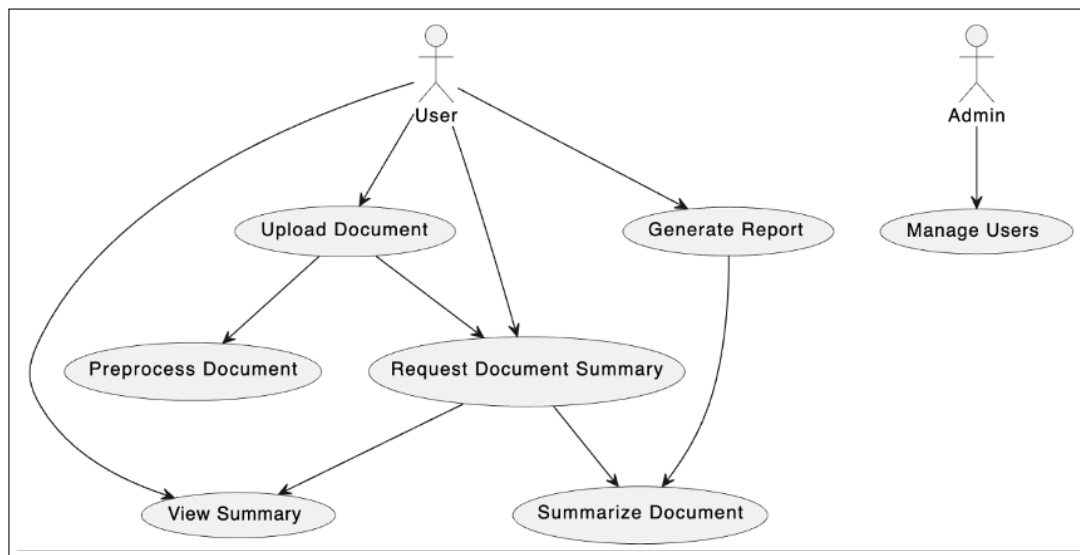


Figure 4.5: UseCase Diagram

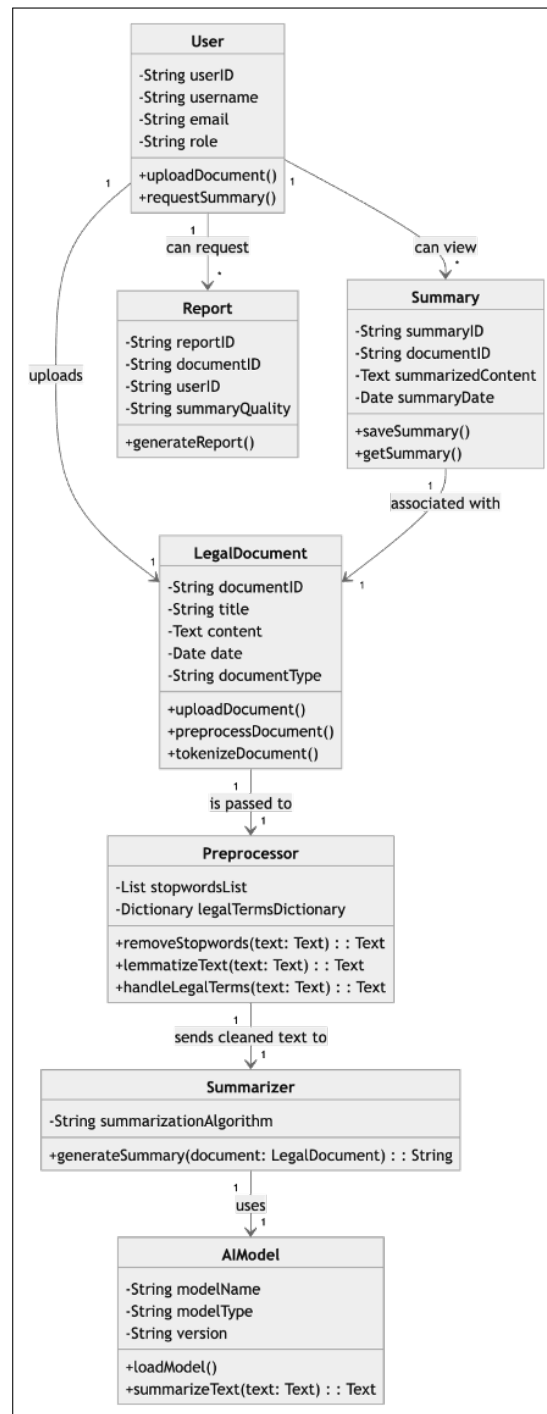


Figure 4.6: Class Diagram

4.6 Sequence Diagram

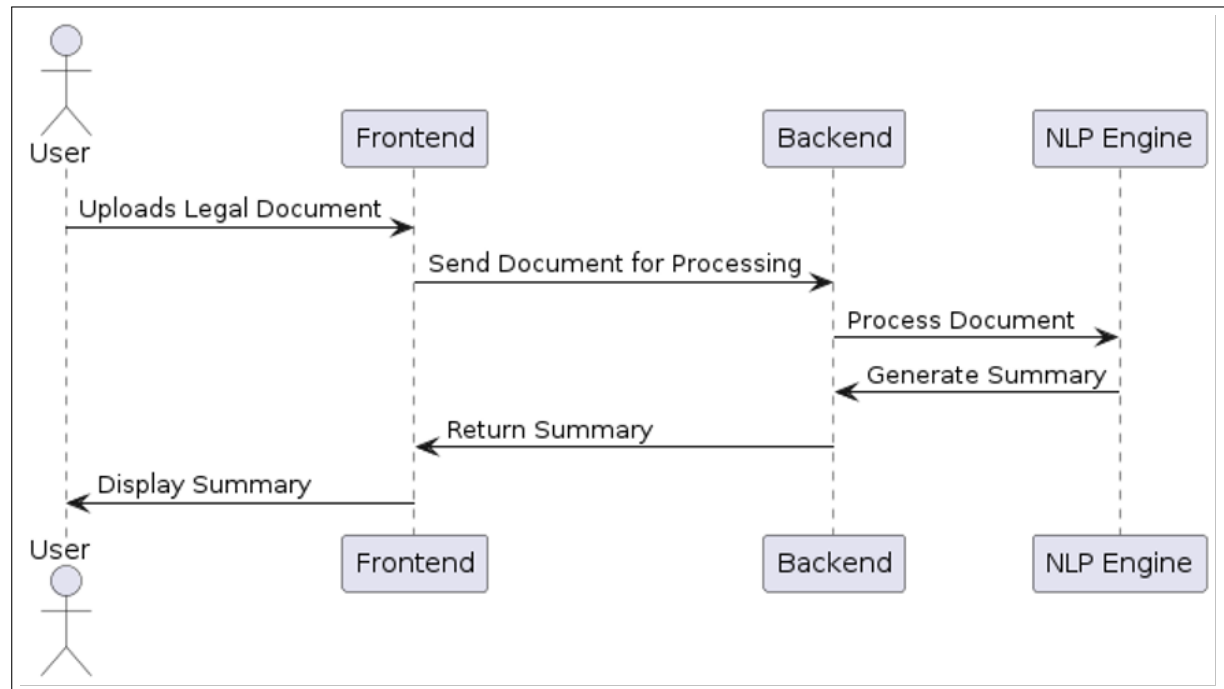


Figure 4.7: Sequence Diagram

CHAPTER 5

PROJECT IMPLEMENTATION

5.1 Overview of Project Modules

The project is centered around the development of a legal document summarization system by fine-tuning a domain-adapted BART model using LoRA (Low-Rank Adaptation). Given the complexity and verbosity of legal documents, a modular approach was adopted to handle different stages of the pipeline systematically. The first module focuses on environment setup, including configuring the hardware (e.g., CUDA for GPU acceleration), initializing random seeds for reproducibility, and loading essential libraries and toolkits.

The second module deals with data ingestion and preprocessing, where raw legal documents and their corresponding summaries are cleaned and tokenized. To address the limitation of transformer models on input length, a sentence-based chunking method is introduced, enabling the model to process long documents without truncating essential content.

The third module constructs a custom dataset compatible with PyTorch, which formats the tokenized documents and summaries into tensors ready for model input. The fourth module involves model initialization, where a BART model pre-trained on legal corpora is injected with LoRA adapters to reduce training complexity and enhance efficiency.

A comprehensive training loop constitutes the fifth module, supporting gradient accumulation, learning rate scheduling, and mixed-precision training. The sixth module handles validation, model checkpointing, and early stopping mechanisms to save the best-performing model.

Evaluation of model performance is managed in the seventh module using ROUGE metrics. Lastly, the eighth module performs hyperparameter optimization through grid search, identifying the best configuration for fine-tuning the model. Each module is isolated yet integrated into a cohesive training and evaluation pipeline.

5.2 Tools and Technologies Used

This project utilizes a wide range of libraries and tools from the Python ecosystem, with PyTorch serving as the core deep learning framework. PyTorch's flexibility and compatibility with GPU computation made it ideal for handling large-scale transformer models. The Hugging Face Transformers library was used to load and fine-tune the BART model, providing high-level APIs for tokenization and training routines.

The PEFT (Parameter-Efficient Fine-Tuning) library was employed to inject LoRA modules into the pre-trained model, reducing the number of trainable parameters and enabling resource-efficient fine-tuning. This is particularly useful

for large models that are otherwise costly to train. Tokenization and sentence segmentation tasks were carried out using the Natural Language Toolkit (NLTK), which offers robust tools for handling English-language text.

The Evaluate library from Hugging Face was used to calculate ROUGE scores, which are industry-standard metrics for summarization quality. Data manipulation and exploration were facilitated by pandas and NumPy, while tqdm was used to track training and evaluation progress with real-time updates.

The experiments were conducted in a CUDA-enabled Kaggle notebook environment, which provided the necessary GPU resources for efficient training. Flash attention and memory-efficient settings were enabled through PyTorch backend configurations to further optimize runtime performance. Overall, the toolchain combined state-of-the-art libraries with practical engineering techniques for scalable model training.

5.3 Algorithm Details

5.3.1 Text Chunking and Preprocessing

Legal documents often exceed the token limits imposed by transformer architectures like BART, which can lead to truncated inputs and loss of critical context. To mitigate this, the input documents and summaries are preprocessed using a chunking approach. Sentence tokenization is applied to split the documents into coherent segments, and a sliding window or greedy accumulation strategy is used to create chunks that stay within the model's maximum input length.

This technique ensures that each chunk contains complete, meaningful sentences and avoids splitting mid-sentence, which could impair comprehension and summarization. The same method is applied to summaries, although they tend to be shorter. The resulting chunks are stored in pairs, maintaining their positional relationship for training.

By breaking down long documents into smaller, manageable parts, we enable the model to process legal content effectively while maintaining contextual relevance. This approach is particularly suited for the legal domain, where precision and completeness are critical.

5.3.2 Dataset Construction

After preprocessing, the chunks are used to construct a custom dataset class compatible with PyTorch's DataLoader. Each instance in the dataset consists of a document chunk prefixed with the token "summarize:" to guide the model's behavior, and its corresponding summary chunk. These inputs and outputs are

tokenized using a domain-specific tokenizer and are padded or truncated to fixed lengths to ensure consistent tensor sizes.

Padding tokens are appropriately masked in the label sequences to prevent them from contributing to the loss during backpropagation. The dataset class returns dictionaries containing input IDs, attention masks, and labels, enabling seamless integration into the training loop.

This modular dataset design abstracts away preprocessing complexity during training and supports efficient batch loading, shuffling, and parallel processing. It also allows for easy extension or modification, such as adding metadata or applying further transformations.

5.3.3 Model Initialization with LoRA

To fine-tune the BART model efficiently, we utilize Low-Rank Adaptation (LoRA), which reduces the number of trainable parameters by injecting trainable rank-decomposed matrices into pre-specified layers of the model. Specifically, the LoRA adapters are added to the query and value projection layers in the attention mechanism of the BART encoder and decoder.

LoRA enables efficient adaptation by training only a small number of additional parameters while freezing the majority of the original model weights. This significantly reduces GPU memory consumption and training time, making it viable to fine-tune large language models even on limited hardware.

The LoRA configuration is customizable, allowing specification of the rank (r), alpha scaling factor, dropout rate, and target modules. Once configured, the modified model is loaded onto the appropriate device (CPU or GPU) for training. This approach maintains the expressive power of the pre-trained BART model while making the fine-tuning process more lightweight.

5.3.4 Training Procedure

The training loop is designed for stability, efficiency, and performance. It uses the AdamW optimizer, which is well-suited for transformer models, and incorporates a learning rate scheduler with a warmup phase to avoid instability in early epochs. To simulate larger batch sizes without exceeding memory constraints, gradient accumulation is employed, allowing the model to accumulate gradients over multiple steps before updating weights.

Mixed-precision training is activated using automatic casting and gradient scaling, significantly speeding up training and reducing memory consumption. Loss values are tracked at each step, and gradient clipping is applied to prevent exploding gradients. After each epoch, the model is evaluated on a validation set to monitor generalization.

If a new best validation loss is observed, the model's parameters are saved as the current best. This ensures that only the most effective model is preserved for evaluation and deployment. The training loop also includes GPU cache clearing and garbage collection to manage memory effectively across epochs.

5.3.5 Evaluation Metrics

The performance of the summarization model is evaluated using ROUGE metrics, which measure the overlap between generated summaries and ground-truth references. Specifically, ROUGE-1 measures unigram overlap, ROUGE-2 captures bigram overlap, and ROUGE-L accounts for the longest common subsequence. These metrics collectively provide a comprehensive picture of the model's summarization quality.

To perform evaluation, a representative sample from the validation set is selected, and the model generates summaries for each input document. These generated summaries are then compared to the actual summaries using the Evaluate library. Stemmer-based matching is used to normalize inflected word forms, improving robustness.

The evaluation results not only provide a numerical assessment of performance but also help identify specific areas of improvement. Higher ROUGE scores indicate better retention of key facts and relevance in the generated summaries, which are crucial in the legal domain where accuracy and completeness are essential.

5.3.6 Hyperparameter Tuning

To optimize the model's performance, hyperparameter tuning is carried out using a grid search approach. The search explores combinations of learning rate, batch size, and the number of training epochs. For each configuration, a new model instance is initialized with LoRA, trained on the dataset, and evaluated using ROUGE metrics.

The tuning process involves reloading and resetting the model to ensure that prior training states do not influence subsequent trials. Performance scores are logged, and the configuration yielding the highest ROUGE-1 score is selected as the best. This best configuration is then used for the final model training.

Hyperparameter tuning is a critical step in model development, as suboptimal settings can lead to poor generalization, overfitting, or inefficient training. By systematically searching the parameter space, we ensure that the model operates under settings that maximize performance while minimizing training cost.

CHAPTER 6

PROJECT PLAN

The project plan outlines the estimates, risks, schedule, and organizational structure for the development of the legal document summarization system. Effective project planning ensures that the goals are met within the given timeline, while considering potential risks and ensuring proper team coordination.

6.1 Project Estimates

The initial estimates provide a rough calculation of the time and resources required to complete the project. Key components include:

- **Time Estimation:** Developing an NLP-based summarization system for legal documents will require approximately 6–8 months. The main phases include:
 - Research and model selection: 1–2 months.
 - Data collection and preprocessing: 1–1.5 months.
 - Model training and testing: 2–3 months.
 - UI/UX design and integration: 1–2 months.
 - Final testing and deployment: 1 month.
- **Resource Estimation:**
 - Access to pre-trained NLP models (e.g., BERT, GPT-3).
 - Legal datasets (e.g., contract databases, case rulings).
 - Cloud computing resources for training large models.
 - Legal domain expertise for validating summarization accuracy.

6.1.1 Reconciled Estimates

Once the initial estimates are reviewed, a more refined version—reconciled estimates—can be provided. These estimates are adjusted based on factors like risk analysis, team expertise, and budget constraints:

- The time required may increase by 10–15% if there are delays in data collection or model tuning.
- Resource requirements might fluctuate depending on the availability of legal datasets and cloud services. We may need more compute power depending on the size and complexity of the NLP model.

6.2 Risk Management

6.2.1 Risk Identification

Key risks associated with the project include:

- **Data availability risk:** Legal documents are often private and difficult to access.
- **Model performance risk:** The model may not perform well on all types of legal documents, particularly with complex clauses or legal jargon.
- **Regulatory and ethical risks:** Sensitive data raises privacy and security concerns.
- **Project delay risk:** Technical issues or miscommunication among team members could lead to delays.

6.2.2 Risk Analysis

Risks are assessed based on their probability and impact:

- **Data availability risk:** High probability, high impact. Mitigation involves using alternative sources or synthetic data.
- **Model performance risk:** Medium probability, high impact. Mitigated through model tuning and expert validation.
- **Regulatory and ethical risks:** Medium probability, medium impact. Adherence to regulations like GDPR mitigates this.
- **Project delay risk:** Medium probability, medium impact. Mitigated through effective communication and tracking.

6.2.3 Risk Mitigation, Monitoring, and Management

Mitigation strategies include:

- **Data availability:** Collaborate with law firms or use synthetic data generation.
- **Model performance:** Rigorous testing and iterative improvement using active learning.

- **Compliance:** Regular consultation with legal experts to ensure compliance.
- **Project tracking:** Employ milestone-based tracking and regular team meetings.

6.3 Project Schedule

6.3.1 Project Task Set

Table 6.1: Project Task Sets

Phase	Task Description
1	Research and model selection
2	Data collection and preprocessing
3	Model training and testing
4	UI/UX design and integration
5	Final testing and deployment

6.3.2 Timeline Chart

A Gantt chart will be used to track project progress. It will visualize the timeline for each task, showing the start and end dates, as well as task dependencies. Regular updates to the chart will help identify and address any delays or bottlenecks.

Table 6.2: Timeline Chart for Project

Task ID	Task	Start Date	End Date
1	Research and model selection	Month 1	Month 2
2	Data collection and preprocessing	Month 2	Month 3
3	Model training and testing	Month 3	Month 6
4	UI/UX design and integration	Month 5	Month 6
5	Final testing and deployment	Month 6	Month 7

6.4 Team Organization

6.4.1 Team Structure

The project team includes the following roles:

- **Project Manager:** Oversees project execution and communication.

- **NLP Developers:** Develop the summarization model.
- **Data Scientists:** Handle dataset collection, preprocessing, and analysis.
- **UI/UX Designers:** Design the system interface.
- **Legal Domain Experts:** Validate summarization accuracy.
- **Testers:** Conduct extensive functionality and accuracy testing.

6.4.2 Management Reporting and Communication

Communication will be maintained through:

- Regular team status meetings.
- Progress reports.
- Project management tools like Jira or Trello.

Each team member reports to the Project Manager, who consolidates updates and communicates with stakeholders.

CHAPTER 7

SOFTWARE TESTING

7.1 Type of Testing

- **Unit Testing:** Each component of the summarization pipeline (e.g., tokenization, model input/output formatting, post-processing) was tested independently.
- **Functional Testing:** End-to-end testing was conducted to ensure that the input legal document produced an appropriate summary with consistent performance.
- **Performance Testing:** The speed of inference and system responsiveness were tested under different loads to ensure timely summary generation.
- **Accuracy Testing:** The summarization output was evaluated against reference summaries using ROUGE scores to determine the system's precision and recall.
- **Usability Testing:** Feedback from legal professionals and non-expert users was collected on the interface and clarity of the summaries.

7.2 Test Cases & Test Results

- **Test Case 1: Input Preprocessing**
Description: Ensure input legal documents are cleaned and tokenized properly.
Expected Result: No malformed tokens, consistent structure.
Result: Passed.
- **Test Case 2: Summary Generation**
Description: System should generate a summary for varied types of legal texts (contracts, rulings, etc.).
Expected Result: Coherent summary that reflects key ideas.
Result: Passed.
- **Test Case 3: Edge Case Handling**
Description: Test with very long or very short legal documents.
Expected Result: Graceful handling with informative summaries.
Result: Passed with warnings (long documents required truncation).
- **Test Case 4: UI Functionality**
Description: Ensure all user interface elements work as expected.
Expected Result: Upload, summarize, download functions operate smoothly.
Result: Passed.

CHAPTER 8

RESULTS

8.1 Outcomes

1. Developed a legal text summarization system capable of generating concise and accurate summaries using the BART transformer model.
2. Achieved the ROUGE-1 score of 53.16, indicating a good match with human-generated summaries across test datasets.
3. Designed a user-friendly web interface for legal professionals and general users to upload documents and receive instant summaries.

8.2 Screen Shots

```
Loading dataset...

Processing data...

100%|██████████| 5624/5624 [02:14<00:00, 41.92it/s]
100%|██████████| 1406/1406 [00:32<00:00, 42.65it/s]

Starting hyperparameter search...

Trying lr=3e-05, batch_size=8, epochs=3

Loading model...

model.safetensors:  0%|          | 0.00/1.63G [00:00<?, ?B/s]

generation_config.json:  0%|          | 0.00/363 [00:00<?, ?B/s]

Applied LoRA to model successfully.
trainable params: 1,179,648 || all params: 407,470,080 || trainable%: 0.2895
```

Figure 8.1: Screenshot of the final results

```
Epoch 1/3

Training: 100%|██████████| 1588/1588 [1:00:17<00:00, 2.28s/it]
Validating: 100%|██████████| 395/395 [05:51<00:00, 1.12it/s]

Train Loss: 3.1408 | Val Loss: 2.8587
Saved best model!

Epoch 2/3

Training: 100%|██████████| 1588/1588 [1:00:50<00:00, 2.30s/it]
Validating: 100%|██████████| 395/395 [05:51<00:00, 1.12it/s]

Train Loss: 2.9114 | Val Loss: 2.7908
Saved best model!
```

Figure 8.2: Screenshot of the final results

```
Epoch 3/3

Training: 100%|██████████| 1588/1588 [1:01:15<00:00, 2.31s/it]
Validating: 100%|██████████| 395/395 [05:52<00:00, 1.12it/s]

Train Loss: 2.8718 | Val Loss: 2.7755
Saved best model!

Downloading builder script: 0%|          | 0.00/6.27k [00:00<?, ?B/s]

Evaluating: 100%|██████████| 50/50 [38:09<00:00, 45.79s/it]

Final ROUGE Scores:
ROUGE-1 Score      : 53.1667%
ROUGE-2 Score      : 33.8370%
ROUGE-L Score      : 29.0582%
Token Precision     : 39.1962%
Token Recall        : 67.8524%
Token F1 Score      : 48.6293%
```

Figure 8.3: Screenshot of the final results

CHAPTER 9

CONCLUSIONS

9.1 Conclusion

While the project is aimed to achieve its primary goal of automating the summarization of legal documents, there are few more opportunities to enhance its functionality and applicability:

9.2 Future Work

1. **Multi-Lingual Support:** Many legal systems operate in languages other than English. Adding support for other major languages would significantly broaden the tool's applicability across different regions and jurisdictions.
2. **Legal Validation and Compliance:** Incorporating a layer of legal validation and compliance would make the system more reliable, allowing it to not just summarize but also validate certain legal clauses or terms, ensuring that the summarized documents comply with regional laws or contracts.
3. **Integration with Legal Databases:** The system could be integrated with legal databases (e.g., Westlaw, LexisNexis) to cross-reference cases, laws, and contracts with the uploaded documents.
4. **Data Privacy and Security:** Handling sensitive legal documents requires utmost attention to privacy and security. Future work will focus on implementing advanced encryption techniques, secure access protocols, and ensuring GDPR compliance or other regional data privacy laws.

9.3 Applications

The **Automatic Legal Document Summarization** project has several real-world applications across different sectors. Some of the most notable applications are:

1. **Law Firms:**
Law firms often handle a large volume of legal documents daily, including contracts, court rulings, and case files. This tool can assist lawyers and paralegals by summarizing these documents, allowing them to focus on higher-level legal tasks.
2. **Corporate Legal Departments:**
Large corporations with in-house legal teams can use the tool to summarize

lengthy contracts, compliance documents, and policies, saving significant time and resources.

3. Government Agencies:

Government agencies that deal with legal regulations, contracts, and interdepartmental agreements can use the system to generate summaries for faster decision-making and documentation processing.

4. Real Estate and Property Law:

Real estate firms often deal with extensive legal documentation related to property transactions, leases, and agreements. The summarization tool can help them quickly understand the key elements of these documents.

5. Financial Institutions:

Banks and other financial institutions manage contracts, loan agreements, and regulatory compliance documents. Summarizing these documents can aid in quicker decision-making and reviewing important terms and conditions.

6. Academic Research:

Legal researchers, students, and academic professionals can use the summarization tool to analyze legal documents, cases, and contracts efficiently, reducing the time spent on manual reading.

7. E-discovery and Litigation Support:

During the discovery process in litigation, legal teams review large volumes of documents. Automatic summarization can help identify key pieces of information more quickly, aiding in the overall litigation strategy.

CHAPTER 10

APPENDIX

10.1 Appendix A

10.1.1 Problem Statement Overview

The task focuses on automatic abstractive summarization of legal documents using a transformer-based model enhanced with parameter-efficient fine-tuning techniques (LoRA) and domain-adapted pretraining (Legal-BART). The goal is to generate coherent, legally sound summaries of long, unstructured judicial texts.

This involves:

- Sentence segmentation and tokenization of large legal texts.
- Mapping high-dimensional language representations.
- Training and evaluating a sequence-to-sequence model (BART) under constraints like input length, coherence, and factual consistency.

10.1.2 Feasibility via Satisfiability

The task transforms a set of natural language tokens (document D) into another set (summary S), preserving semantics. Mathematically, this can be viewed as checking the satisfiability of a function:

$$f : D \rightarrow S$$

Subject to:

(Semantic entailment)	$\forall \phi \in S, D \models \phi$
(Length constraint)	$ S < D $
(Consistency)	$\neg \exists \psi \in S \text{ such that } D \not\models \psi$

While exact semantic equivalence is undecidable in natural language, the satisfiability of such a function f is approximated using neural language models. Feasibility is probabilistic and depends on:

- The quality and quantity of training data.
- The architecture and parameterization of the model.
- The optimization process.

10.1.3 Complexity Classification

10.1.3.1 Problem Reduction Perspective

The summarization task can be reduced from the **Minimum Text Compression Problem**, known to be NP-Hard, due to:

- **Semantic constraints:** Capturing all critical legal content.
- **Combinatorial nature:** The number of possible summaries grows exponentially with the document length.

The training objective minimizes the cross-entropy loss over possible summaries. This objective simulates a non-deterministic search through a vast hypothesis space, constrained by:

- Length,
- Legal coherence,
- Lexical and syntactic fluency.

10.1.3.2 Approximate Heuristics

Modern transformer-based models like BART with LoRA act as heuristic solvers:

- **Greedy/Beam decoding:** Efficient search approximation.
- **Pretrained priors:** Legal-BART leverages domain knowledge.
- **Decoding constraints:** Maximum token length limits the search.

These methods do not ensure optimal solutions but provide **practical feasibility** under limited time and compute.

10.1.3.3 NP-Completeness

Consider the decision version:

"Is there a summary S under k tokens that captures all legal arguments from document D ?"

This can be reduced from the **Minimum Set Cover Problem** — a well-known NP-Complete problem. Hence, legal summarization is:

NP-Hard, and potentially NP-Complete under the decision formulation.

10.1.4 Modern Algebraic Insight

The model operates as a function:

$$f(x; \theta)$$

Where:

- $x \in \Sigma^*$ is an input sentence in legal language,
- $\theta \in \mathbb{R}^n$ is a learned parameter vector,
- f is a composition of non-linear transformer blocks.

We seek:

$$f(x; \theta) = y$$

where y is the desired summary. This mapping is not solvable algebraically but is learned iteratively through stochastic optimization.

Feasibility depends on:

- Existence of a learnable manifold mapping $x \mapsto y$,
- Model capacity (depth and width),
- Availability of domain-specific training data,
- Generalization bounds (e.g., via VC-dimension in statistical learning theory).

10.1.5 Practical Implication

Despite being NP-Hard in theory, the task is feasible in practice due to:

- Pretrained transformer models like Legal-BART,
- Efficient fine-tuning techniques such as LoRA,
- Domain-specific preprocessing (chunking, tokenization),
- Practical dataset sizes (e.g., ~ 500 examples).

10.2 Appendix B

10.2.1 Survey Paper

Journal Name: BULLETIN FOR TECHNOLOGY AND HISTORY

Status: Published

Volume and Issue: Volume 24 Issue 11 2024

Title: Automatic Summarization of Legal Documents

Abstract:

Automatic summarization of legal documents is a rapidly evolving field that leverages natural language processing (NLP) techniques to extract key information from lengthy and complex legal documents. Legal documents are often lengthy, complex, and filled with jargon that can be challenging to understand. Traditional methods of summarization require manual extraction of key information, which is time-consuming and prone to human error. There is a need for an automated system that can efficiently and accurately summarize legal documents, providing clear and concise information that highlights the key terms, obligations, rights, and conditions stipulated within the document.

10.2.2 Implementation Paper

Conference Name: 9th International Conference on Control Communication, Computing & Automation (ICCUBEA-2025)

Status: Submitted/In Review Process

Title: Fine-Tuning BART with LoRA for Summarization of Indian Judicial Case Documents

Abstract:

In this paper, we will explore fine-tuning the BART model with Low-Rank Adaptation (LoRA) on judicial documents from the Indian government. Indian judicial documents are lengthy and intricate, making summarization a challenging task. We have introduced the chunking mechanism to manage long legal texts within model limits. Our approach uses standard crossentropy loss for training and assesses performance with ROUGE and token-level precision recall metrics on a pre-processed legal dataset. A structured grid search optimizes hyperparameters by analyzing the effects of learning rate, batch size, and epochs on model outcomes. This strategy improves the efficiency and quality of summary generation for legal texts. In general, our rese

10.3 Appendix C

Plagiarism Report of project report.

Document Information

Analyzed document	be_Report.pdf (D210181418)
Submitted	2025-04-26 10:55:00 UTC+02:00
Submitted by	Pranjali
Submitter email	pranjalianshu@gmail.com
Similarity	6%
Analysis address	pranjalianshu.pict@analysis.arkund.com

Sources included in the report

SA	20037727.pdf Document 20037727.pdf (D209348323)		1
W	URL: https://dr.ntu.edu.sg/bitstream/10356/168487/2/Amended_thesis_LinXiang.pdf Fetched: 2025-04-26 10:57:00		1
W	URL: https://WWW.coursehero.com/file/236179255/BEproject-2nd-sem-report-3-removedpdf/ Fetched: 2025-04-26 10:56:00		1
SA	Pune Institute of Computer Technology / be_Report (2).pdf Document be_Report (2).pdf (D172571887) Submitted by: sssonawane@pict.edu Receiver: sssonawane.pict@analysis.arkund.com		4
W	URL: http://arxiv.org/pdf/2502.10632 Fetched: 2025-04-26 10:56:00		1
W	URL: https://dailytrojan.com/wp-content/uploads/2023/10/Grad-School-Guide-Supplement-Oct-24-2023-HYPERLINKS.pdf Fetched: 2025-04-26 10:56:00		1
SA	Pune Institute of Computer Technology / be_Report.pdf Document be_Report.pdf (D191199386) Submitted by: kangrade@pict.edu Receiver: kangrade.pict@analysis.arkund.com		2
W	URL: https://link.springer.com/article/10.1007/s10506-024-09414-w?code=c3ca1958-a591-4abb-b69c-441a673123f5&error=cookies_not_supported Fetched: 2025-04-26 10:57:00		2
W	URL: https://arxiv.org/pdf/2404.04302 Fetched: 2025-04-26 10:56:00		1
W	URL: https://www.mdpi.com/1099-4300/27/4/351 Fetched: 2025-04-26 10:56:00		1

W	URL: https://sie.engineering.arizona.edu/faculty-staff/faculty/tomas-cerny Fetched: 2025-04-26 10:56:00	 1
W	URL: https://arxiv.org/pdf/2404.13087 Fetched: 2025-04-26 10:56:00	 1
W	URL: https://hrcak.srce.hr/ojs/index.php/jahr/article/download/32515/17426/150713 Fetched: 2025-04-26 10:56:00	 1
SA	erazimova_report.pdf Document erazimova_report.pdf (D205210326)	 1
W	URL: https://ijrpr.com/uploads/V5ISSUE11/IJRPR35537.pdf Fetched: 2025-04-26 10:56:00	 1

Entire Document

A PROJECT REPORT ON Automatic Summarization of Legal Documents
SUBMITTED
TO THE SAVITRIBAI PHULE UNIVERSITY, PUNE IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE
AWARD OF
THE DEGREE OF BACHELOR OF ENGINEERING (Computer Engineering) SUBMITTED BY
Aayush Goyal Exam No: B400050311 Arnav Firke Exam No: B400050388
Bhairu Gavhane
Exam No: B400050397 Shrinidhi Kulkarni Exam No: B400050463
DEPARTMENT
OF COMPUTER ENGINEERING
Pune Institute of Computer Technology Dhankawadi, Pune - 411043 SAVITRIBAI PHULE PUNE UNIVERSITY 2024-2025
CERTIFICATE This is to certify that the project report entitled
Automatic Summarization of Legal Documents
Submitted by Aayush Goyal Exam No: B400050311 Arnav Firke Exam No: B400050388
Bhairu Gavhane
Exam No: B400050397 Shrinidhi Kulkarni Exam No: B400050463
is a
bonafide student of this institute and the work has been
carried out by him/her under
the supervision of Prof. P. P. Joshi
and it is approved for
the partial fulfillment of the requirement of
Savitribai Phule Pune University,
for the award
of the degree of Bachelor of Engineering (Computer Engineering).
Prof.
P. P. Joshi Dr. Geetanjali V. Kale Internal Guide Head Dept. of Computer
Engg. Dept.
of Computer Engg. Dr.
S. T. Gandhe Principal Pune Institute of Computer Technology Place:Pune Date:
ACKNOWLEDGEMENT
It
gives us great pleasure
in presenting the project report on 'Automatic

CHAPTER 11

REFERENCES

- [1] CanLII. (n.d.). Canadian Legal Information Institute. <https://www.canlii.org/en/>
- [2] Manor, L. (n.d.). Legal summarization dataset. GitHub. <https://www.github.com/lauramanor/legal-summarization>
- [3] Zenodo. (n.d.). Legal summarization dataset. <https://www.zenodo.org/records/4595826>
- [4] SATYAJIT1910. (n.d.). Indian legal document summarization dataset. GitHub. <https://github.com/SATYAJIT1910/ILDS>
- [5] AustLII. (n.d.). Australian Legal Information Institute. <https://www.austlii.edu.au/>
- [6] Law Reports. (n.d.). Law Reports. <https://www.lawreports.co.uk/>
- [7] Denninger. (n.d.). EUR-Lex summary dataset. Hugging Face. <http://huggingface.co/datasets/denninger/eur-lex-sum>
- [8] Bhattacharya, P., Poddar, S., & Rudra, K. (n.d.). Incorporating domain knowledge for extractive summarization of legal case documents. arXiv. <https://arxiv.org/pdf/2106.15876>
- [9] Ghosh, S., Dutta, M., & Das, T. (n.d.). Indian legal text summarization: A text normalization-based approach. arXiv. <https://arxiv.org/pdf/2206.06238>
- [10] Klaus, S., Van Hecke, R., Djafari Naini, K., Sengor Altingovde, I., Bernab-Moreno, J., & Herrera-Viedma, E. (2022). Summarizing legal regulatory documents using transformers. ACM Digital Library. <https://web.archive.org/web/20220712231042id/https://dl.acm.org/doi/pdf/10.1145/3477495.3531872>
- [11] Chalkidis, I., Fergadiotis, M., Malakasiotis, P., Aletras, N., & Androutsopoulos, I. (2020). LEGAL-BERT: The muppets straight out of law school. Findings of EMNLP. <https://aclanthology.org/2020.findings-emnlp.261.pdf>
- [12] Mamakas, D., Tsotsi, P., Androutsopoulos, I., & Chalkidis, I. (2022). Processing long legal documents with pre-trained transformers: Modding LegalBERT and Longformer. arXiv. <https://arxiv.org/pdf/2211.00974v2>

- [13] Caballero, E. Q., Rahman, M., Cerny, T., Rivas, P., & Bejarano, G. (2022). Study of question answering on legal software documents using BERT-based models. *LatinX in AI Research*. <https://research.latinxinai.org/papers/naacl/2022/pdf/paper09.pdf>
- [14] Polsley, S., Jhunjhunwala, P., & Huang, R. (2016). CaseSummarizer: A system for automated summarization of legal texts. *COLING 2016*. https://people.engr.tamu.edu/huangrh/papers/coling16_caseSummarizer.pdf
- [15] Farzindar, A., & Lapalme, G. (n.d.). Letsum, an automatic legal text summarizing system. *ResearchGate*. https://www.researchgate.net/publication/228980166_Letsum_an_automatic_legal_text_summarizing_system
- [16] Burman, A., & Bradford, E. (n.d.). Building an optimized algorithm that provides summaries of legal documents. *ResearchGate*. https://www.researchgate.net/publication/365294672_Building_an_optimized_algorithm_that_provides_summaries_of_legal_documents
- [17] Soneji, S., Hoising, M., Koujalgi, S., & Dodge, J. (2024). Demystifying legalese: An automated approach for summarizing and analyzing overlaps in privacy policies and terms of service. *arXiv*. <https://arxiv.org/pdf/2404.13087>
- [18] Althammer, S., Askari, A., Verberne, S., & Hanbury, A. (2021). Leveraging dense retrieval and summarization-based re-ranking for case law retrieval. *arXiv*. <https://arxiv.org/pdf/2108.03937>
- [19] Nguyen, H., & Ding, J. (2023). Keyword-based augmentation method to enhance abstractive summarization for legal documents. *ACM Digital Library*. <https://dl.acm.org/doi/pdf/10.1145/3594536.3595120>
- [20] Jain, D., Borah, M. D., & Biswas, A. (2021). Summarization of legal documents: Where are we now and the way forward. *Computer Science Review*, 40, 100388. <https://doi.org/10.1016/j.cosrev.2021.100388>
- [21] Yadav, D., Katna, R., Yadav, A. K., & Morato, J. (2022). Feature-based automatic text summarization methods: A comprehensive state-of-the-art survey. *IEEE Access*, 10, 133981-134003. <https://doi.org/10.1109/ACCESS.2022.3231016>